

인증과 세션

(Authentication and Session)

인증(Authentication)

- 기본 인증 메커니즘 이해

- 인증(Authentication)은 사용자가 누구인지 확인하는 과정
- 클라이언트(웹 브라우저)와 서버 간의 상호작용에서 중요함
- HTTP 헤더를 사용하여 인증 정보를 서버에 전달함

- 인증을 위한 HTTP 헤더

- HTTP 헤더는 클라이언트와 서버 간 데이터 교환 시 중요한 역할을 함
- 'Authorization' 헤더는 인증 정보를 담음
- 예시: Authorization:Basic<인코딩된 문자열>
- 여기서 <인코딩된 문자열>은 '아이디:비밀번호'를 Base64로 인코딩한 값임

- FastAPI에서의 기본 인증 구현

- FastAPI와 HTTPBasic을 사용하여 인증 시스템을 구현함

코드 예시

```
from fastapi import FastAPI, Depends, HTTPException
from fastapi.security import HTTPBasic, HTTPBasicCredentials

app = FastAPI()
security = HTTPBasic()

def get_current_username(credentials: HTTPBasicCredentials = Depends(security)):
    if credentials.username != "alice" or credentials.password != "password":
        raise HTTPException(status_code=401, detail="Unauthorized")
    return credentials.username

@app.get("/users/me")
def read_current_user(username: str = Depends(get_current_username)):
    return {"username": username}
```

- HTTPBasic는 기본 인증 방식을 구현함
- get_current_username 함수는 사용자 이름과 비밀번호를 검증함
- 잘못된 인증 정보의 경우 HTTP 401 오류를 반환함
- /users/me 엔드포인트는 인증된 사용자만 접근할 수 있음

실행 방법

- 다음 curl 명령어를 사용하여 인증 테스트를 수행할 수 있음
 - `curl -X 'GET' "http://127.0.0.1:8000/users/me" -u alice:password`
- FastAPI를 활용한 기본 인증 구현 방법을 통해, 사용자의 요청이 올바른 인증 정보를 갖고 있는지 검증하고, 그에 따라 적절한 응답을 제공할 수 있음

세션(Session)

- 세션의 역할

- 웹에서 상태를 유지하는 중요한 방법
- HTTP 프로토콜의 상태 없음(stateless) 문제를 해결함

- 세션의 작동 원리

- 첫 요청 시 서버는 클라이언트에 고유한 세션 ID를 발급
- 세션 ID는 클라이언트 측 쿠키에 저장됨
- 클라이언트의 재요청 시, 쿠키에 저장된 세션 ID를 서버에 전송
- 서버는 세션 ID를 통해 클라이언트를 식별하고, 정보를 저장하거나 불러옴

FastAPI 에서 세션 사용하기

- FastAPI와 Starlette 세션 활용
 - FastAPI는 Starlette의 기능을 상속받아 세션 관리 기능을 제공함
 - SessionMiddleware는 Starlette의 미들웨어로, 세션 관리를 수행함
- 세션 미들웨어 설정
 - FastAPI 앱에 SessionMiddleware를 추가하여 세션 기능을 활성화함
 - secret_key는 세션 데이터를 암호화하는데 사용되며, 안전하게 관리되어야 함

코드 예시

```
from fastapi import FastAPI, Request
from starlette.middleware.sessions import SessionMiddleware

app = FastAPI()
app.add_middleware(SessionMiddleware, secret_key="your-secret-key")
```

- secret_key 관리
 - 환경 변수를 사용하여 secret_key를 안전하게 관리함
 - key 길이는 충분히 길고 복잡하게 설정함
 - 필요에 따라 키 관리 서비스(AWS Secrets Manager, HashiCorp Vault 등)를 활용
 - 주기적인 키 갱신을 고려

세션 데이터 설정 및 조회

- Request 객체를 사용하여 세션에 데이터를 저장하고 조회
- 세션 설정 코드 예시

```
@app.post("/set/")
async def set_session(request: Request):
    request.session["username"] = "john"
    return {"message": "Session value set"}
```

- 세션 조회 코드 예시

```
@app.get("/get/")
async def get_session(request: Request):
    username = request.session.get("username", "Guest")
    return {"username": username}
```

- /set/ 경로에서 세션에 username을 설정
- /get/ 경로에서 세션에서 username을 조회함
- async 키워드는 비동기 작업을 고려하여 사용되며, 현재 코드에서는 필수는 아님

실행하기

- curl을 사용한 테스트 방법

- 세션 설정 요청: /set/ 엔드포인트로 세션을 설정함
 - -c cookies.txt : 쿠키를 cookies.txt 파일에 저장함

```
curl -X 'POST' 'http://127.0.0.1:8000/set/' -i -c cookies.txt
```

- 세션 값 가져오기 요청: /get/ 엔드포인트로 세션 값을 조회
 - -b cookies.txt : cookies.txt 파일에서 쿠키를 읽어 요청에 포함함

```
curl -X 'GET' 'http://127.0.0.1:8000/get/' -i -b cookies.txt
```

cookies.txt 파일

```
# Netscape HTTP Cookie File
# https://curl.se/docs/http-cookies.html
# This file was generated by libcurl! Edit at your own risk.

#HttpOnly_127.0.0.1 FALSE / FALSE 1699777921 session
eyJ1c2VybmFtZSI6ICJqb2huIn0=.ZT4YgQ.tfGTpfIjNTemvImR2pvgLIdfqVU
```

- 쿠키 파일의 구조와 플래그 설명

- #HttpOnly_127.0.0.1 : 쿠키의 도메인을 나타냄
 - HttpOnly 쿠키는 클라이언트 측 스크립트(예: JavaScript)에서 접근할 수 없으며, 오직 HTTP 요청을 통해서만 서버에 전송됨
 - 이는 쿠키를 통한 크로스 사이트 스크립팅(XSS) 공격을 방지하는 데 도움이 됨
- FALSE : 첫 번째 FALSE는 쿠키가 전송되어야 하는 경우에 대한 보안 연결(HTTPS)의 필요성을 나타냄. 여기서 FALSE는 쿠키가 HTTPS가 아닌 일반 HTTP 연결은 통해서도 전송될 수 있음을 의미함
- / : 이 부분은 쿠키의 “Path” 속성을 나타내며, 쿠키가 전송되어야 하는 URL 경로를 지정함. 여기서 / 는 도메인의 모든 경로에 대해 쿠키가 유효함을 의미

cookies.txt 파일

```
# Netscape HTTP Cookie File
# https://curl.se/docs/http-cookies.html
# This file was generated by libcurl! Edit at your own risk.

#HttpOnly_127.0.0.1 FALSE / FALSE 1699777921 session
eyJ1c2VybmFtZSI6ICJqb2huIn0=.ZT4YgQ.tfGTpfIjNTemvImR2pvgLIdfqVU
```

● 쿠키 파일의 구조와 플래그 설명

- FALSE : 두 번째 FALSE는 쿠키의 Secure 속성과 관련이 있음. 이 필드가 TRUE일 경우, 쿠키는 오직 HTTPS 연결을 통해서만 전송됨. 여기서 FALSE는 쿠키가 보안 연결이 아닌 상태에서도 전송될 수 있음을 나타냄
- 1699777921 : 이 숫자는 쿠키의 만료 시간을 UNIX 타임스탬프(1970년 1월 1일부터 경과한 초)로 나타냄. 쿠키가 이 시간에 도달하면 더 이상 유효하지 않게 됨
- session : 쿠키의 이름임
- 이후 값 : 세션 데이터의 암호화된 값임

FastAPI 세션을 이용한 로그인 예제 최종 코드

```
from fastapi import FastAPI, Request, HTTPException

from starlette.middleware.sessions import SessionMiddleware

app = FastAPI()

# 세션 미들웨어 추가, 'verysecret'는 실제 사용 시 안전한 키로 교체해야 합니다.
app.add_middleware(SessionMiddleware, secret_key="verysecret")

@app.post("/login/")
async def login(request: Request, username: str, password: str):
    if username == "john" and password == "1234":
        request.session["username"] = username
        return {"message": "Successfully logged in"}
    else:
        raise HTTPException(status_code=401, detail="Invalid credentials")

@app.get("/dashboard/")
async def dashboard(request: Request):
    username = request.session.get("username")
    if not username:
        raise HTTPException(status_code=401, detail="Not authorized")
    return {"message": f"Welcome to the dashboard, {username}"}
```

코드 구성

- 로그인 예제 코드

- FastAPI 어플리케이션에 세션 관리 기능을 추가하고, 로그인 및 인증된 사용자만 접근할 수 있는 대시보드를 구현함

- 코드 구성

- 세션 미들웨어 설정
 - SessionMiddleware를 앱에 추가하여 세션 관리를 활성화함
 - secret_key는 세션 데이터를 암호화하는 데 사용됨
- 로그인 엔드포인트(/login/)
 - 사용자가 제공한 username과 password를 검증함
 - 유효한 자격 증명의 경우, 사용자 이름을 세션에 저장함
- 대시보드 엔드포인트(/dashboard/)
 - 세션에서 사용자 이름을 조회함
 - 로그인하지 않은 사용자는 접근할 수 없음

실행하기

- 로그인 실행

- 로그인을 시도하고 세션 쿠키를 cookies.txt 파일에 저장함

```
curl -X "POST" "http://127.0.0.1:8000/login/?username=john&password=1234" -i -c cookies.txt
```

- 성공 시, 로그인 성공 메시지를 반환

- 대시보드 접근 테스트

- 로그인 후 대시보드에 접근. 세션 쿠키는 cookies.txt에서 가져옴

```
curl -X "GET" "http://127.0.0.1:8000/dashboard/" -i -b cookies.txt
```

- 인증된 사용자는 대시보드 환영 메시지를 받음